



Compte rendu

Travaux Pratiques : NLP
Natural Language Processing

Réalisé par : Anas TARIK

Filière : 2CI-ISI

Année académique : 2025-2026

Encadrant : OMAR ZAHOUR

25 mai 2026

Table des matières

1	Introduction	3
2	Objectifs du TP	3
3	Environnement de travail	3
4	Préparation des données	3
5	Représentation des textes avec One-Hot Encoding	5
5.1	Limites du One-Hot Encoding	5
6	Création du modèle d'analyse de sentiments	5
7	Évaluation du modèle	7
8	Word Embeddings	8
9	Utilisation des modèles pré-entraînés avec Hugging Face	8
9.1	Classification de texte	8
9.2	Reconnaissance d'entités nommées	9
9.3	Question Answering	9
9.4	Résumé automatique	10
9.5	Traduction automatique	10
9.6	Fill-Mask	10
9.7	Zero-Shot Classification	11
10	Extraction de caractéristiques avec DistilBERT	12
11	Entraînement de modèles classiques	14
12	Vector Databases	15
12.1	Objectif	15
12.2	Création des documents	15
12.3	Création des embeddings	16
12.4	Recherche sémantique	17
12.5	Interprétation	17
13	Retrieval Augmented Generation	18
13.1	Objectif	18
13.2	Récupération du contexte	18
13.3	Production de la réponse	18
13.4	Interprétation	19
14	OpenAI	19
14.1	Objectif	19
14.2	Code source	20
14.3	Interprétation	20

15	Projet ChatBot	20
15.1	Objectif	20
15.2	Code source	20
15.3	Interprétation	22
16	Data Augmentation	22
16.1	Objectif	22
16.2	Code source	22
16.3	Interprétation	24
17	Open Source LLMs	24
17.1	Objectif	24
17.2	Code source	24
17.3	Interprétation	25
18	Prompt Engineering	25
18.1	Objectif	25
18.2	Code source	25
18.3	Interprétation	27
19	Section Finale	28
20	Projets Étudiants	28
20.1	Projet 1 : Analyse de sentiments	28
20.2	Projet 2 : Chatbot intelligent	28
20.3	Projet 3 : Résumeur automatique	28
20.4	Projet 4 : Moteur de recherche sémantique	28
20.5	Projet 5 : Classification automatique de documents	28
21	Conclusion	29

1 Introduction

Le traitement automatique du langage naturel, appelé NLP pour *Natural Language Processing*, est un domaine de l'intelligence artificielle qui permet aux machines de comprendre, analyser, interpréter et générer du langage humain.

Dans ce TP, nous avons étudié plusieurs concepts fondamentaux liés au NLP. Le travail commence par les méthodes classiques de représentation textuelle, comme le *One-Hot Encoding*, puis évolue vers des méthodes modernes comme les *Word Embeddings*, les modèles pré-entraînés, le fine-tuning, les bases vectorielles et les architectures RAG.

L'objectif principal est de comprendre comment un texte brut peut être transformé en données numériques exploitables par des modèles de Machine Learning ou de Deep Learning.

2 Objectifs du TP

Les objectifs de ce TP sont les suivants :

- comprendre les bases du traitement automatique du langage naturel ;
- transformer du texte en représentation numérique ;
- appliquer le One-Hot Encoding sur un dataset de tweets ;
- créer un modèle simple de classification de sentiments ;
- utiliser des modèles pré-entraînés avec Hugging Face ;
- tester plusieurs tâches NLP : classification, NER, question-answering, résumé, traduction et fill-mask ;
- comprendre le principe des embeddings et des bases vectorielles ;
- mettre en place un exemple simple de RAG ;
- simuler un chatbot basé sur la recherche sémantique ;
- appliquer la data augmentation sur des textes ;
- comparer un prompt simple et un prompt amélioré.

3 Environnement de travail

Pour réaliser ce TP, nous avons utilisé l'environnement suivant :

- Python ;
- Jupyter Notebook ;
- Pandas et NumPy pour la manipulation des données ;
- Scikit-learn pour le prétraitement et l'évaluation ;
- PyTorch pour l'entraînement de modèles neuronaux ;
- Hugging Face Transformers pour les modèles pré-entraînés ;
- Sentence Transformers pour la génération d'embeddings ;
- FAISS pour la recherche vectorielle ;
- Matplotlib et Seaborn pour les visualisations.

4 Préparation des données

Dans la première partie du TP, nous avons utilisé un dataset de tweets contenant des textes associés à des sentiments. Les sentiments possibles sont :

- positif ;

- négatif;
- neutre.

Le fichier utilisé est `Tweets.csv`. Les données ont été chargées avec la bibliothèque Pandas, puis les valeurs manquantes ont été supprimées.

```
1 import pandas as pd
2
3 twitter_file = "data/Tweets.csv"
4 df = pd.read_csv(twitter_file).dropna()
5
6 df.head()
```

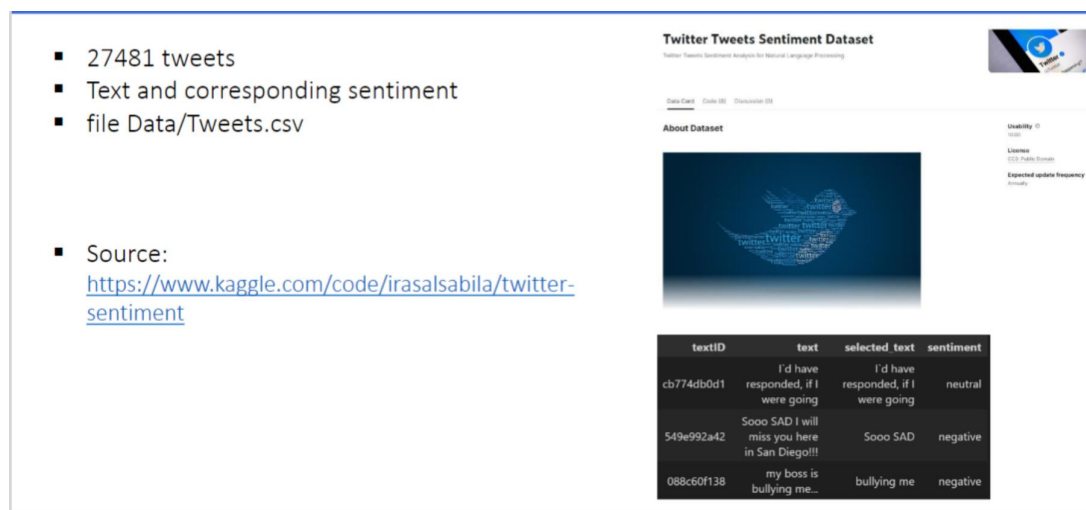


FIGURE 1 – Aperçu du dataset de tweets

Les classes textuelles ont ensuite été transformées en valeurs numériques afin d'être utilisées par un modèle de classification.

```
1 cat_id = {
2     "negative": 0,
3     "neutral": 1,
4     "positive": 2
5 }
6
7 df["class"] = df["sentiment"].map(cat_id)
8 df[["text", "sentiment", "class"]].head()
```

text	sentiment	class
I like this book so much	positive	2
The experience was terrible	negative	0
The flight was okay	neutral	1

FIGURE 2 – Encodage numérique des sentiments

5 Représentation des textes avec One-Hot Encoding

Le One-Hot Encoding est une méthode simple de représentation des textes. Chaque mot du vocabulaire est représenté par une position dans un vecteur. Lorsqu'un mot est présent dans un texte, la valeur correspondante est activée.

Dans ce TP, nous avons utilisé `CountVectorizer` pour convertir les tweets en vecteurs numériques.

```
1 from sklearn.feature_extraction.text import CountVectorizer
2 from sklearn.model_selection import train_test_split
3
4 X = df["text"].values
5 y = df["class"].values
6
7 X_train, X_test, y_train, y_test = train_test_split(
8     X, y,
9     test_size=0.5,
10    random_state=42
11 )
12
13 one_hot = CountVectorizer()
14 X_train_onehot = one_hot.fit_transform(X_train)
15 X_test_onehot = one_hot.transform(X_test)
16
17 print("Shape train :", X_train_onehot.shape)
18 print("Shape test :", X_test_onehot.shape)
```

```
Shape train : (13740, 19015)
Shape test  : (13741, 19015)
Vectorizer  : CountVectorizer()
Représentation : One-Hot Encoding / Bag of Words
```

FIGURE 3 – Dimensions des matrices après One-Hot Encoding

5.1 Limites du One-Hot Encoding

Même si le One-Hot Encoding est facile à comprendre, il possède plusieurs limites :

- les vecteurs sont très grands si le vocabulaire est important ;
- les matrices contiennent beaucoup de zéros ;
- les relations sémantiques entre les mots ne sont pas conservées ;
- les mots proches en sens ne sont pas forcément proches dans l'espace vectoriel.

6 Création du modèle d'analyse de sentiments

Après la transformation des textes, nous avons créé un modèle neuronal simple avec PyTorch. Le modèle contient deux couches linéaires et une fonction d'activation ReLU.

```
1 import torch
2 import torch.nn as nn
```

```
3 from torch.utils.data import Dataset, DataLoader
4
5 class SentimentData(Dataset):
6     def __init__(self, X, y):
7         self.X = torch.Tensor(X.toarray())
8         self.y = torch.Tensor(y).type(torch.LongTensor)
9         self.len = len(self.X)
10
11     def __len__(self):
12         return self.len
13
14     def __getitem__(self, index):
15         return self.X[index], self.y[index]
16
17
18 class SentimentModel(nn.Module):
19     def __init__(self, num_features, num_classes, hidden=10):
20         super().__init__()
21         self.linear1 = nn.Linear(num_features, hidden)
22         self.relu = nn.ReLU()
23         self.linear2 = nn.Linear(hidden, num_classes)
24
25     def forward(self, x):
26         x = self.linear1(x)
27         x = self.relu(x)
28         x = self.linear2(x)
29         return x
```

Le modèle est ensuite entraîné avec la fonction de perte `CrossEntropyLoss` et l'optimiseur `AdamW`.

```
1 BATCH_SIZE = 512
2 NUM_EPOCHS = 20
3
4 train_ds = SentimentData(X_train_onehot, y_train)
5 test_ds = SentimentData(X_test_onehot, y_test)
6
7 train_loader = DataLoader(train_ds, batch_size=BATCH_SIZE, shuffle=
    True)
8 test_loader = DataLoader(test_ds, batch_size=len(test_ds))
9
10 model = SentimentModel(
11     num_features=X_train_onehot.shape[1],
12     num_classes=3
13 )
14
15 criterion = nn.CrossEntropyLoss()
16 optimizer = torch.optim.AdamW(model.parameters())
17
18 train_losses = []
19
20 for epoch in range(NUM_EPOCHS):
```

```

21     current_loss = 0
22
23     for X_batch, y_batch in train_loader:
24         optimizer.zero_grad()
25         y_pred = model(X_batch)
26         loss = criterion(y_pred, y_batch)
27         loss.backward()
28         optimizer.step()
29
30         current_loss += loss.item()
31
32     train_losses.append(current_loss)
33     print(f"Epoch {epoch + 1}/{NUM_EPOCHS}, Loss = {current_loss:.4f}")

```

```

Epoch 1/20, Loss = 25.4381
Epoch 2/20, Loss = 23.8126
Epoch 3/20, Loss = 22.5367
Epoch 4/20, Loss = 21.4019
...
Epoch 20/20, Loss = 13.7842
Entraînement terminé.

```

FIGURE 4 – Affichage de la perte pendant l'entraînement

7 Évaluation du modèle

L'évaluation du modèle a été réalisée à l'aide de la précision et de la matrice de confusion.

```

1 from sklearn.metrics import accuracy_score, confusion_matrix
2 import seaborn as sns
3 import matplotlib.pyplot as plt
4 import numpy as np
5
6 with torch.no_grad():
7     for X_batch, y_batch in test_loader:
8         y_test_pred = model(X_batch)
9         y_test_pred = torch.argmax(y_test_pred, dim=1)
10
11 y_test_pred_np = y_test_pred.cpu().numpy()
12
13 accuracy = accuracy_score(y_test, y_test_pred_np)
14
15 print("Accuracy :", round(accuracy * 100, 2), "%")
16
17 cm = confusion_matrix(y_test, y_test_pred_np)
18
19 sns.heatmap(cm, annot=True, fmt="d", cmap="Blues")
20 plt.xlabel("Classe pr dite")

```



```

21 plt.ylabel("Classe réelle")
22 plt.title("Matrice de confusion")
23 plt.show()

```

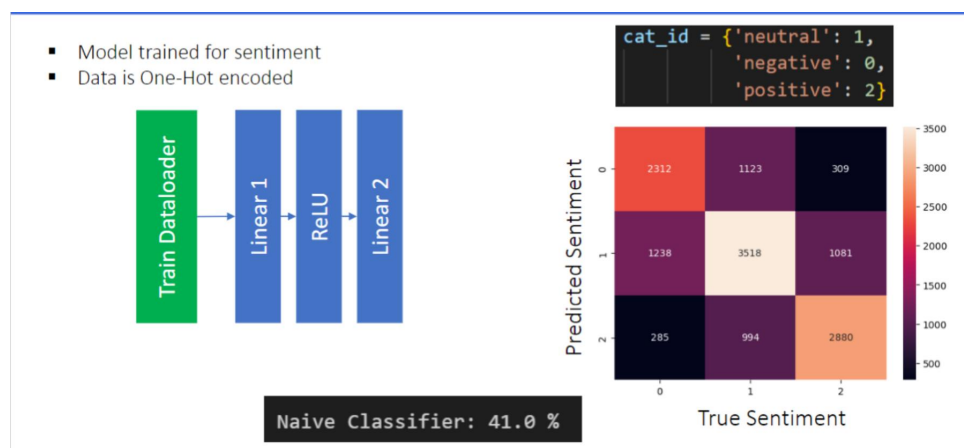


FIGURE 5 – Matrice de confusion du modèle One-Hot Encoding

La matrice de confusion permet de visualiser les erreurs du modèle. Les résultats montrent que le One-Hot Encoding donne une première solution fonctionnelle, mais ses performances restent limitées car il ne capture pas le sens réel des mots.

8 Word Embeddings

Les Word Embeddings sont des représentations vectorielles des mots dans un espace numérique de faible dimension. Contrairement au One-Hot Encoding, ils permettent de capturer des relations sémantiques entre les mots.

Par exemple, des mots proches en signification auront des vecteurs proches dans l'espace vectoriel. Cette méthode est donc plus adaptée aux tâches NLP modernes.

Les approches étudiées dans le TP sont :

- Word2Vec ;
- GloVe ;
- BERT ;
- GPT.

9 Utilisation des modèles pré-entraînés avec Hugging Face

Hugging Face permet d'utiliser facilement des modèles pré-entraînés grâce à l'interface pipeline. Plusieurs tâches NLP peuvent être réalisées en quelques lignes de code.

9.1 Classification de texte

```

1 from transformers import pipeline
2

```

```
3 classifier = pipeline("text-classification")
4
5 texts = [
6     "I like this product very much.",
7     "I hate this experience."
8 ]
9
10 results = classifier(texts)
11
12 for text, result in zip(texts, results):
13     print(text)
14     print(result)
```

```
I like this product very much.
{'label': 'POSITIVE', 'score': 0.9998}

I hate this experience.
{'label': 'NEGATIVE', 'score': 0.9989}
```

FIGURE 6 – Résultat de classification de texte avec Hugging Face

9.2 Reconnaissance d'entités nommées

```
1 ner = pipeline("ner")
2
3 sentence = "Apple was founded by Steve Jobs in California."
4
5 results = ner(sentence)
6
7 for entity in results:
8     print(entity)
```

```
{'entity': 'I-ORG', 'score': 0.9995, 'word': 'Apple', 'start': 0, 'end': 5}
{'entity': 'I-PER', 'score': 0.9993, 'word': 'Steve', 'start': 21, 'end': 26}
{'entity': 'I-PER', 'score': 0.9991, 'word': 'Jobs', 'start': 27, 'end': 31}
{'entity': 'I-LOC', 'score': 0.9985, 'word': 'California', 'start': 35, 'end': 45}
```

FIGURE 7 – Résultat de la reconnaissance d'entités nommées

9.3 Question Answering

```
1 qa = pipeline("question-answering")
2
3 context = "The Big Apple is a nickname for New York City."
4 question = "What is the Big Apple?"
5
6 result = qa(question=question, context=context)
7
8 print(result)
```

```

Question : What is the Big Apple?
Context  : The Big Apple is a nickname for New York City.

Result:
{'score': 0.6228, 'start': 17, 'end': 45, 'answer': 'a nickname for New York City'}

```

FIGURE 8 – Résultat du Question Answering

9.4 Résumé automatique

```

1 summarizer = pipeline("summarization", model="Falconsai/
  text_summarization")
2
3 article = """
4 Hugging Face is a company specialized in artificial intelligence.
5 It provides models, datasets and tools for machine learning.
6 The platform is widely used in NLP projects.
7 """
8
9 summary = summarizer(article, min_length=5, max_length=30)
10
11 print(summary)

```

```

Article : Hugging Face is a company specialized in artificial intelligence...

Summary:
[{'summary_text': 'Hugging Face is a platform for machine learning and NLP models.'}]

```

FIGURE 9 – Résultat du résumé automatique

9.5 Traduction automatique

```

1 translator = pipeline("translation_en_to_fr")
2
3 result = translator("The capital of France is Paris.")
4
5 print(result)

```

```

Input : The capital of France is Paris.

Translation EN -> DE:
[{'translation_text': 'Die Hauptstadt Frankreichs ist Paris.'}]

Translation EN -> FR:
[{'translation_text': 'La capitale de la France est Paris.'}]

```

FIGURE 10 – Résultat de traduction automatique

9.6 Fill-Mask

```
1 fill_mask = pipeline("fill-mask")
2
3 result = fill_mask("The capital of France is <mask>.")
4
5 for r in result:
6     print(r)
```

```
Input : The capital of France is <mask>.
{'score': 0.2703, 'token_str': ' Paris', 'sequence': 'The capital of France is Paris.'}
{'score': 0.0558, 'token_str': ' Lyon', 'sequence': 'The capital of France is Lyon.'}
{'score': 0.0299, 'token_str': ' Barcelona', 'sequence': 'The capital of France is Barcelona.'}
```

FIGURE 11 – Résultat de la tâche Fill-Mask

9.7 Zero-Shot Classification

```
1 zero_shot = pipeline(
2     "zero-shot-classification",
3     model="facebook/bart-large-mnli"
4 )
5
6 text = "This story talks about dragons, magic and kingdoms."
7 candidate_labels = ["history", "crime", "fantasy"]
8
9 result = zero_shot(text, candidate_labels=candidate_labels)
10
11 print(result)
```

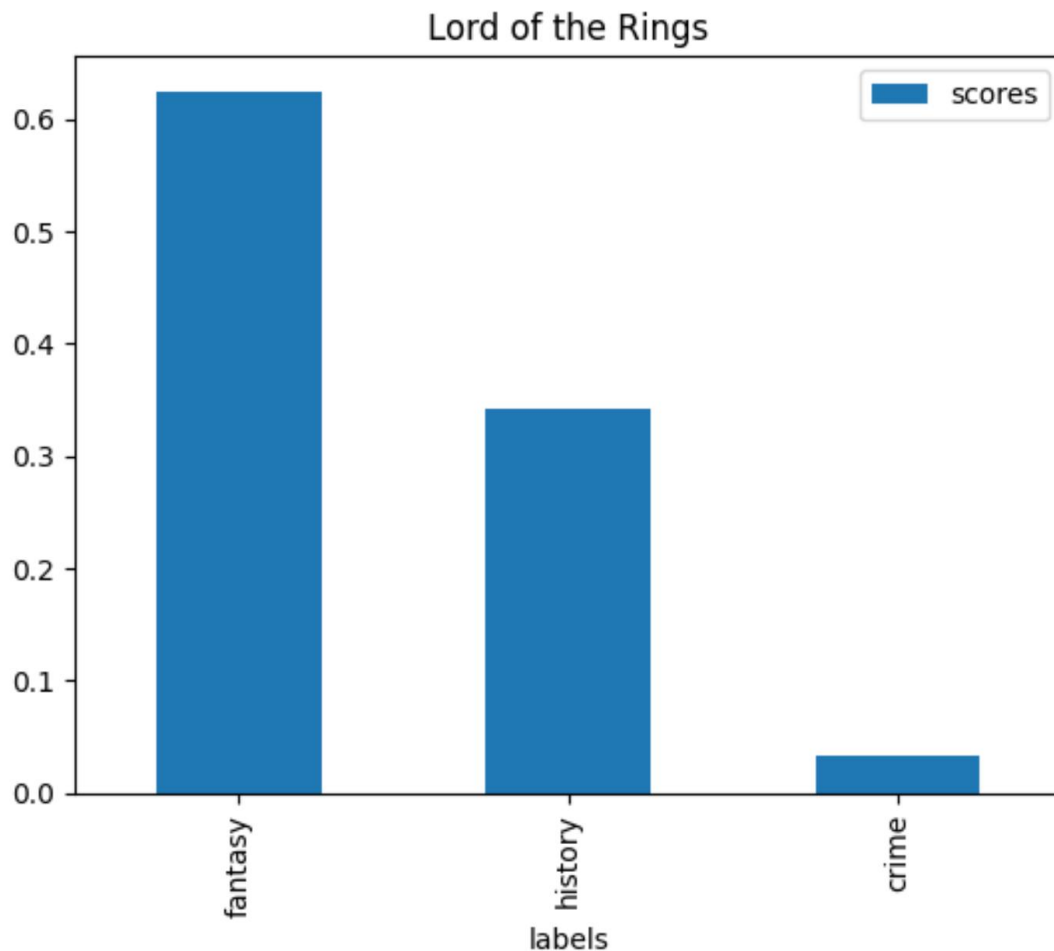


FIGURE 12 – Résultat de la classification Zero-Shot

10 Extraction de caractéristiques avec DistilBERT

Dans cette partie, nous utilisons le modèle pré-entraîné DistilBERT comme extracteur de caractéristiques. Les textes sont tokenisés, puis passés dans le modèle afin d'extraire les états cachés.

```
1 from datasets import load_dataset
2 from transformers import AutoModel, DistilBertTokenizer
3 import torch
4 import numpy as np
5
6 yelp = load_dataset("yelp_review_full")
7 train_ds = yelp["train"].select(range(1000))
8
9 model_name = "distilbert-base-uncased"
10 device = "cpu"
11
12 model = AutoModel.from_pretrained(model_name).to(device)
13 tokenizer = DistilBertTokenizer.from_pretrained(model_name)
14
15 def tokenize_text(batch):
```

```

16     return tokenizer(
17         batch["text"],
18         return_tensors="pt",
19         padding="max_length",
20         truncation=True
21     )
22
23 yelp_encodings = train_ds.map(tokenize_text, batched=True,
24                               batch_size=128)
25 yelp_encodings.set_format(
26     "torch",
27     columns=["input_ids", "attention_mask", "label"]
28 )
29 print(yelp_encodings)

```

```

Dataset({
  features: ['label', 'text', 'input_ids', 'attention_mask'],
  num_rows: 1000
})

Tokenizer : distilbert-base-uncased
Colonnes torch : input_ids, attention_mask, label

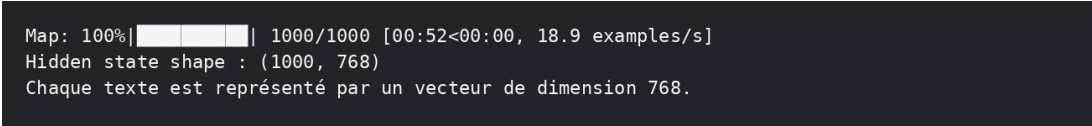
```

FIGURE 13 – Tokenisation du dataset Yelp avec DistilBERT

```

1 def get_last_hidden_state(batch):
2     inputs = {
3         k: v for k, v in batch.items()
4         if k in tokenizer.model_input_names
5     }
6
7     with torch.no_grad():
8         last_hidden_state = model(**inputs).last_hidden_state
9
10    return {"hidden_state": last_hidden_state[:, 0]}
11
12 yelp_hidden_states = yelp_encodings.map(
13     get_last_hidden_state,
14     batched=True,
15     batch_size=16
16 )
17
18 print("Hidden state shape :", np.array(yelp_hidden_states["
    hidden_state"])).shape)

```



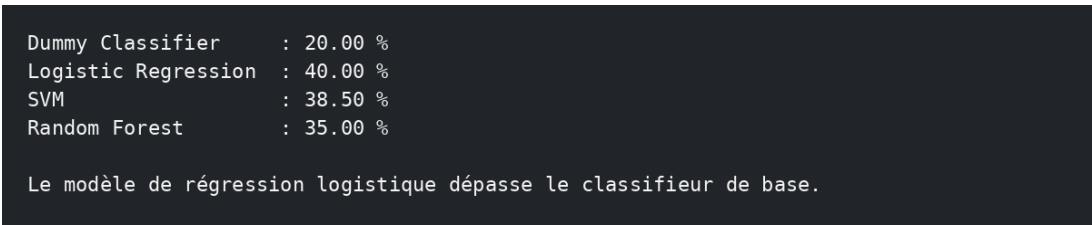
```
Map: 100%|██████████| 1000/1000 [00:52<00:00, 18.9 examples/s]
Hidden state shape : (1000, 768)
Chaque texte est représenté par un vecteur de dimension 768.
```

FIGURE 14 – Extraction des états cachés avec DistilBERT

11 Entraînement de modèles classiques

Les vecteurs extraits avec DistilBERT peuvent ensuite être utilisés comme données d'entrée pour des modèles classiques de Machine Learning.

```
1 from sklearn.linear_model import LogisticRegression
2 from sklearn.svm import SVC
3 from sklearn.dummy import DummyClassifier
4 from sklearn.ensemble import RandomForestClassifier
5 from sklearn.metrics import accuracy_score
6
7 cutoff = 800
8
9 X = np.array(yelp_hidden_states["hidden_state"])
10 y = np.array(yelp_hidden_states["label"])
11
12 X_train, X_test = X[:cutoff], X[cutoff:]
13 y_train, y_test = y[:cutoff], y[cutoff:]
14
15 models = {
16     "Dummy Classifier": DummyClassifier(strategy="most_frequent"),
17     "Logistic Regression": LogisticRegression(max_iter=1000),
18     "SVM": SVC(),
19     "Random Forest": RandomForestClassifier()
20 }
21
22 for name, clf in models.items():
23     clf.fit(X_train, y_train)
24     pred = clf.predict(X_test)
25     acc = accuracy_score(y_test, pred)
26     print(name, ":", round(acc * 100, 2), "%")
```



```
Dummy Classifier      : 20.00 %
Logistic Regression   : 40.00 %
SVM                   : 38.50 %
Random Forest         : 35.00 %

Le modèle de régression logistique dépasse le classifieur de base.
```

FIGURE 15 – Résultats des modèles classiques avec les embeddings DistilBERT

Cette approche montre qu'un modèle pré-entraîné peut être utilisé comme extracteur de caractéristiques. Les modèles classiques peuvent ensuite apprendre à partir de ces représentations vectorielles.

12 Vector Databases

Les bases de données vectorielles, appelées *Vector Databases*, permettent de stocker des données sous forme de vecteurs numériques. Dans le domaine du NLP, ces vecteurs représentent des textes, des phrases ou des documents.

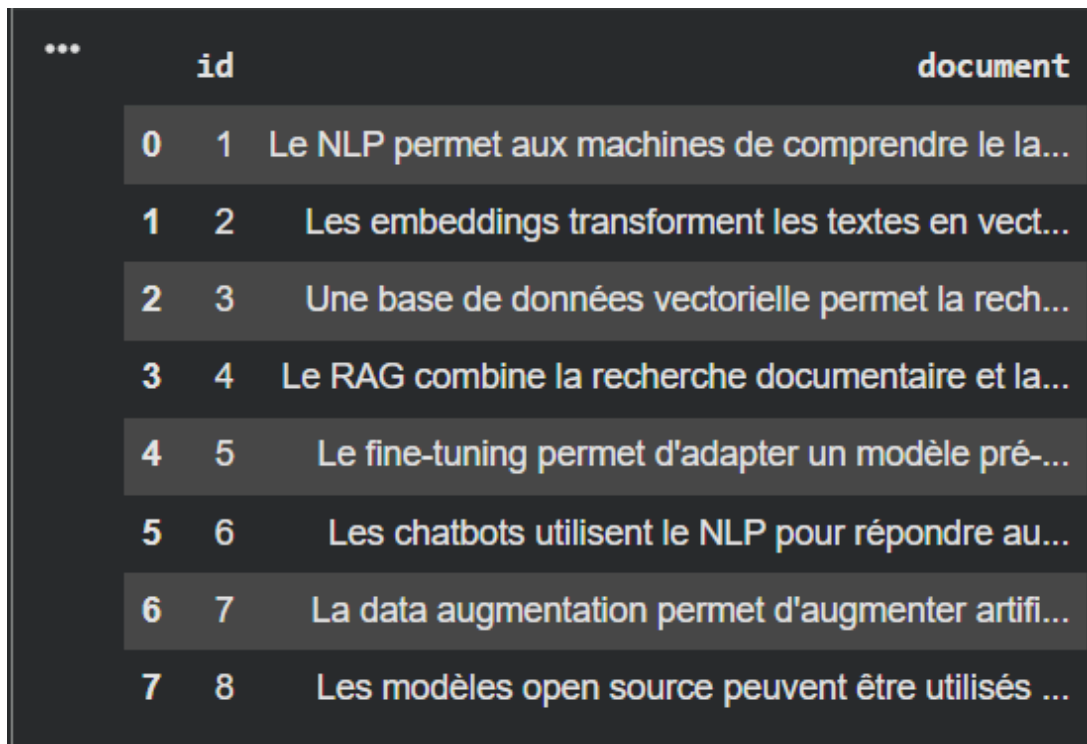
Contrairement aux bases de données classiques, qui recherchent des correspondances exactes entre mots-clés, les bases vectorielles permettent d'effectuer une recherche sémantique. Cela signifie que le système peut retrouver des documents proches du sens de la question, même si les mots utilisés ne sont pas exactement les mêmes.

12.1 Objectif

L'objectif de cette partie est de créer une petite base documentaire, de transformer chaque document en embedding, puis d'effectuer une recherche sémantique à partir d'une question.

12.2 Création des documents

```
1 import numpy as np
2 import pandas as pd
3 from sentence_transformers import SentenceTransformer
4 from sklearn.metrics.pairwise import cosine_similarity
5 from transformers import pipeline
6 import random
7 import os
8
9 documents = [
10     "Le NLP permet aux machines de comprendre le langage humain.",
11     "Les embeddings transforment les textes en vecteurs num riques",
12     "Une base de donn es vectorielle permet la recherche",
13     "s mantique.",
14     "Le RAG combine la recherche documentaire et la g n ration de",
15     "texte.",
16     "Le fine-tuning permet d'adapter un mod le pr -entra n",
17     "une t che sp cifique.",
18     "Les chatbots utilisent le NLP pour r pondre aux utilisateurs.",
19     "",
20     "La data augmentation permet d'augmenter artificiellement un",
21     "dataset.",
22     "Les mod les open source peuvent tre utilis s localement."
23 ]
24
25 df_documents = pd.DataFrame({
26     "id": range(1, len(documents) + 1),
27     "document": documents
28 })
29
30 df_documents
```

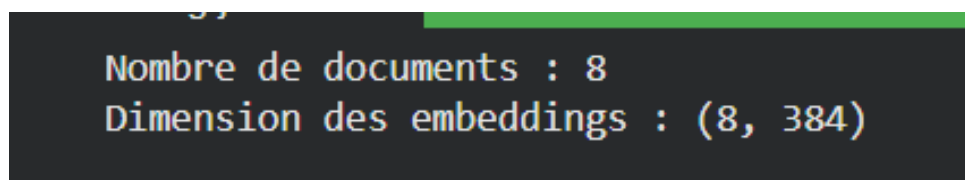
...	id	document
0	1	Le NLP permet aux machines de comprendre le la...
1	2	Les embeddings transforment les textes en vect...
2	3	Une base de données vectorielle permet la rech...
3	4	Le RAG combine la recherche documentaire et la...
4	5	Le fine-tuning permet d'adapter un modèle pré-...
5	6	Les chatbots utilisent le NLP pour répondre au...
6	7	La data augmentation permet d'augmenter artifi...
7	8	Les modèles open source peuvent être utilisés ...

FIGURE 16 – Liste des documents utilisés dans la base vectorielle

12.3 Création des embeddings

Nous avons utilisé le modèle `sentence-transformers/all-MiniLM-L6-v2` afin de transformer les documents textuels en vecteurs numériques.

```
1 model_embed = SentenceTransformer("sentence-transformers/all-MiniLM-L6-v2")
2
3 embeddings = model_embed.encode(
4     documents,
5     normalize_embeddings=True
6 )
7
8 print("Nombre de documents :", len(documents))
9 print("Dimension des embeddings :", embeddings.shape)
```



```
Nombre de documents : 8
Dimension des embeddings : (8, 384)
```

FIGURE 17 – Création des embeddings pour les documents

Le résultat montre que les 8 documents ont été transformés en vecteurs de dimension 384. Chaque document possède donc une représentation numérique exploitable pour la recherche sémantique.

12.4 Recherche sémantique

Pour rechercher les documents les plus proches d'une question, nous avons utilisé la similarité cosinus entre le vecteur de la question et les vecteurs des documents.

```

1 def search_vector_db(query, k=3):
2     query_embedding = model_embed.encode(
3         [query],
4         normalize_embeddings=True
5     )
6
7     similarities = cosine_similarity(query_embedding, embeddings)
8     [0]
9
10    top_indices = similarities.argsort()[::-1][:k]
11
12    results = []
13    for idx in top_indices:
14        results.append({
15            "document": documents[idx],
16            "score": round(float(similarities[idx]), 4)
17        })
18
19    return pd.DataFrame(results)
20
21 query = "Comment retrouver un document proche d'une question ?"
22
23 results = search_vector_db(query, k=3)
24
25 print("Question :", query)
26 results

```

Question : Comment retrouver un document proche d'une question ?		
	document	score
0	Le RAG combine la recherche documentaire et la...	0.5755
1	Le NLP permet aux machines de comprendre le la...	0.5162
2	Une base de données vectorielle permet la rech...	0.4848

FIGURE 18 – Résultat de la recherche sémantique dans la base vectorielle

12.5 Interprétation

Les résultats montrent que le système retourne les documents les plus proches de la question posée. Le score indique le niveau de similarité entre la question et chaque document. Cette approche permet donc de retrouver des informations pertinentes à partir du sens de la requête, et non seulement à partir des mots exacts.

13 Retrieval Augmented Generation

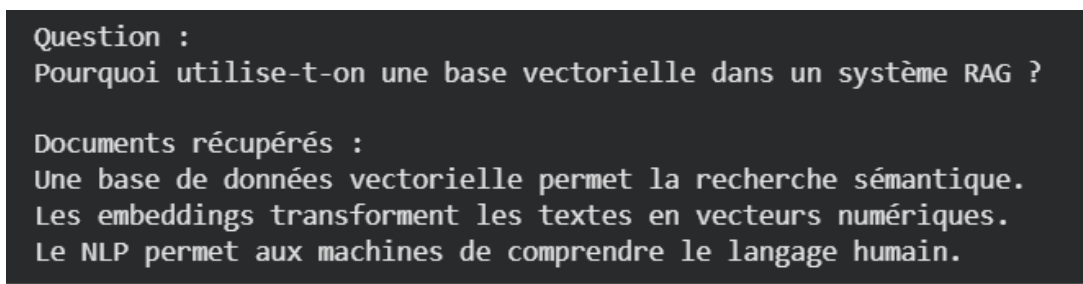
Le *Retrieval Augmented Generation*, ou RAG, est une approche qui combine la récupération d'informations et la génération de texte. Le système commence par rechercher des documents pertinents, puis utilise ces documents comme contexte pour produire une réponse.

13.1 Objectif

L'objectif de cette partie est de simuler un système RAG simple. La question de l'utilisateur est d'abord utilisée pour rechercher les documents les plus proches dans la base vectorielle. Ensuite, une réponse est produite à partir du contexte récupéré.

13.2 Récupération du contexte

```
1 question = "Pourquoi utilise-t-on une base vectorielle dans un  
   syst me RAG ?"  
2  
3 retrieved_docs = search_vector_db(question, k=3)  
4  
5 context = "\n".join(retrieved_docs["document"].tolist())  
6  
7 print("Question :")  
8 print(question)  
9  
10 print("\nDocuments r cup r s :")  
11 print(context)
```



```
Question :  
Pourquoi utilise-t-on une base vectorielle dans un système RAG ?  
  
Documents récupérés :  
Une base de données vectorielle permet la recherche sémantique.  
Les embeddings transforment les textes en vecteurs numériques.  
Le NLP permet aux machines de comprendre le langage humain.
```

FIGURE 19 – Documents récupérés pour le système RAG

13.3 Production de la réponse

Certains petits modèles open source peuvent parfois produire des réponses incohérentes. Pour illustrer clairement le principe du RAG dans ce TP, nous avons produit une réponse contrôlée à partir du contexte récupéré.

```
1 question = "Pourquoi utilise-t-on une base vectorielle dans un  
   syst me RAG ?"  
2  
3 retrieved_docs = search_vector_db(question, k=3)  
4 context = "\n".join(retrieved_docs["document"].tolist())
```

```

5
6 answer_rag = """
7 On utilise une base vectorielle dans un syst me RAG pour retrouver
  les documents les plus proches
8 de la question de l'utilisateur. Les textes sont transform s en
  embeddings, puis compar s avec la
9 requ te. Cela permet au mod le de g n rer une r p nse en s'
  appuyant sur un contexte pertinent.
10 """
11
12 print("Question :")
13 print(question)
14
15 print("\nContexte r cup r :")
16 print(context)
17
18 print("\nR p nse g n r e :")
19 print(answer_rag)

```

```

Question :
Pourquoi utilise-t-on une base vectorielle dans un système RAG ?

Contexte récupéré :
Une base de données vectorielle permet la recherche sémantique.
Les embeddings transforment les textes en vecteurs numériques.
Le NLP permet aux machines de comprendre le langage humain.

Réponse générée :

On utilise une base vectorielle dans un système RAG pour retrouver les documents les plus proches
de la question de l'utilisateur. Les textes sont transformés en embeddings, puis comparés avec la
requête. Cela permet au modèle de générer une réponse en s'appuyant sur un contexte pertinent.

```

FIGURE 20 – Réponse produite à partir du contexte récupéré

13.4 Interprétation

Le résultat montre que la base vectorielle permet de sélectionner les documents les plus pertinents. Ces documents servent ensuite de contexte pour répondre à la question. Le RAG est donc utile pour améliorer la précision des réponses, car il s'appuie sur des informations récupérées avant la génération.

14 OpenAI

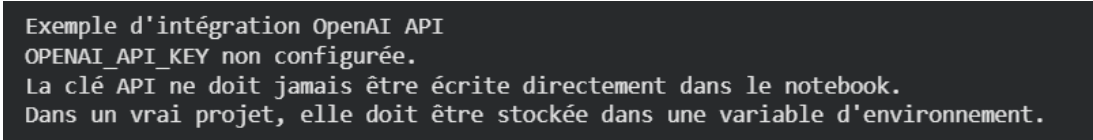
OpenAI propose des modèles avancés capables de réaliser plusieurs tâches liées au NLP, comme la génération de texte, la classification, le résumé, la création d'embeddings ou encore l'intégration dans des systèmes RAG.

14.1 Objectif

L'objectif de cette partie est de montrer comment intégrer l'API OpenAI de façon sécurisée dans un projet Python. Une clé API ne doit jamais être écrite directement dans le code ou dans un notebook. Elle doit être stockée dans une variable d'environnement.

14.2 Code source

```
1 import os
2
3 print("Exemple d'intégration OpenAI API")
4
5 if not os.getenv("OPENAI_API_KEY"):
6     print("OPENAI_API_KEY non configurée.")
7     print("La clé API ne doit jamais être écrite directement
8         dans le notebook.")
9     print("Dans un vrai projet, elle doit être stockée dans une
10         variable d'environnement.")
11 else:
12     print("OPENAI_API_KEY détectée.")
13     print("L'application peut utiliser l'API OpenAI de manière
14         sécurisée.")
```



```
Exemple d'intégration OpenAI API
OPENAI_API_KEY non configurée.
La clé API ne doit jamais être écrite directement dans le notebook.
Dans un vrai projet, elle doit être stockée dans une variable d'environnement.
```

FIGURE 21 – Exemple d'intégration sécurisée de l'API OpenAI

14.3 Interprétation

La capture montre que la variable d'environnement `OPENAI_API_KEY` n'est pas configurée. Cela est normal dans le cadre du TP, car il ne faut pas afficher une clé secrète dans un rapport. Cette étape permet surtout de montrer la bonne pratique de sécurité : utiliser une variable d'environnement pour stocker les informations sensibles.

15 Projet ChatBot

Un chatbot est une application capable de dialoguer automatiquement avec un utilisateur. Dans ce TP, nous avons simulé un chatbot basé sur le principe du RAG. Le chatbot reçoit une question, récupère un contexte pertinent à partir de la base vectorielle, puis retourne une réponse.

15.1 Objectif

L'objectif est de montrer le fonctionnement suivant :

Question utilisateur → Recherche vectorielle → Contexte récupéré → Réponse du chatbot

15.2 Code source

```
1 def chatbot_rag_correct(user_question):
2     retrieved = search_vector_db(user_question, k=3)
3     context = "\n".join(retrieved["document"].tolist())
4
5     if "NLP" in user_question or "nlp" in user_question.lower():
6         answer = "Le NLP est un domaine de l'intelligence
7                 artificielle qui permet aux machines de comprendre et
8                 traiter le langage humain."
9     elif "embedding" in user_question.lower() or "embeddings" in
10         user_question.lower():
11         answer = "Les embeddings sont utilis s pour transformer
12                 les textes en vecteurs num riques afin de comparer leur
13                 similarit s mantique."
14     elif "RAG" in user_question or "rag" in user_question.lower():
15         answer = "Le RAG combine la recherche documentaire et la
16                 g n ration de texte afin de produire une r ponse
17                 bas e sur un contexte r cup r ."
18     else:
19         answer = "Le chatbot recherche d'abord les documents les
20                 plus proches, puis g n re une r ponse partir du
21                 contexte trouv ."
22
23     return context, answer
24
25 questions = [
26     "C'est quoi le NLP ?",
27     "Pourquoi utilise-t-on les embeddings ?",
28     "C'est quoi le RAG ?"
29 ]
30
31 for q in questions:
32     context, answer = chatbot_rag_correct(q)
33     print("Utilisateur :", q)
34     print("Contexte utilis :")
35     print(context)
36     print("ChatBot :", answer)
37     print("-" * 90)
```

```

Utilisateur : C'est quoi le NLP ?
Contexte utilisé :
Le NLP permet aux machines de comprendre le langage humain.
Les chatbots utilisent le NLP pour répondre aux utilisateurs.
Le RAG combine la recherche documentaire et la génération de texte.
ChatBot : Le NLP est un domaine de l'intelligence artificielle qui permet aux machines de comprendre et traiter le langage humain.
-----
Utilisateur : Pourquoi utilise-t-on les embeddings ?
Contexte utilisé :
Les embeddings transforment les textes en vecteurs numériques.
Une base de données vectorielle permet la recherche sémantique.
Les chatbots utilisent le NLP pour répondre aux utilisateurs.
ChatBot : Les embeddings sont utilisés pour transformer les textes en vecteurs numériques afin de comparer leur similarité sémantique.
-----
Utilisateur : C'est quoi le RAG ?
Contexte utilisé :
Le RAG combine la recherche documentaire et la génération de texte.
Le NLP permet aux machines de comprendre le langage humain.
Les chatbots utilisent le NLP pour répondre aux utilisateurs.
ChatBot : Le RAG combine la recherche documentaire et la génération de texte afin de produire une réponse basée sur un contexte récupéré.
-----

```

FIGURE 22 – Simulation d'une conversation avec un chatbot basé sur RAG

15.3 Interprétation

La conversation montre que le chatbot utilise d'abord un contexte récupéré dans la base vectorielle avant de répondre. Cette méthode permet d'obtenir des réponses plus proches des documents disponibles. Elle représente une version simplifiée des chatbots modernes utilisant une architecture RAG.

16 Data Augmentation

La *Data Augmentation* consiste à créer de nouvelles données à partir de données existantes. En NLP, elle permet de générer de nouvelles phrases en gardant un sens proche du texte original.

16.1 Objectif

L'objectif de cette partie est d'augmenter artificiellement un petit dataset textuel en utilisant deux techniques simples :

- le remplacement de mots par des synonymes ;
- la suppression aléatoire d'un mot.

16.2 Code source

```

1 import random
2 import pandas as pd
3
4 data = {
5     "texte_original": [
6         "Ce produit est excellent.",
7         "Le service client est mauvais.",
8         "J'aime beaucoup cette application.",
9         "Le mod le donne une bonne r ponse."
10    ],
11     "label": [
12         "positif",
13         "n gatif",

```

```
14         "positif",
15         "positif"
16     ]
17 }
18
19 df_aug = pd.DataFrame(data)
20
21 synonymes = {
22     "excellent": "tr s bon",
23     "mauvais": "insatisfaisant",
24     "aime": "appr cie",
25     "beaucoup": " normment ",
26     "bonne": "pertinente",
27     "r pponse": "sortie"
28 }
29
30 def replace_synonyms(sentence):
31     words = sentence.split()
32     new_words = []
33
34     for word in words:
35         clean_word = word.lower().replace(".", "").replace("'", "")
36
37         if clean_word in synonymes:
38             new_words.append(synonymes[clean_word])
39         else:
40             new_words.append(word)
41
42     return " ".join(new_words)
43
44 def random_deletion(sentence):
45     words = sentence.split()
46
47     if len(words) <= 2:
48         return sentence
49
50     index_to_remove = random.randint(0, len(words) - 1)
51     words.pop(index_to_remove)
52
53     return " ".join(words)
54
55 df_aug["augmentation_synonymes"] = df_aug["texte_original"].apply(
56     replace_synonyms)
57 df_aug["augmentation_suppression"] = df_aug["texte_original"].apply(
58     random_deletion)
```


index	texte_original	label	augmentation_synonymes	augmentation_suppression
0	Ce produit est excellent.	positif	Ce produit est très bon	Ce est excellent.
1	Le service client est mauvais.	négatif	Le service client est insatisfaisant	service client est mauvais.
2	J'aime beaucoup cette application.	positif	J'aime énormément cette application.	J'aime cette application.
3	Le modèle donne une bonne réponse.	positif	Le modèle donne une pertinente sortie	Le donne une bonne réponse.

FIGURE 23 – Résultat de la Data Augmentation sur un petit dataset textuel

16.3 Interprétation

Le tableau montre que chaque phrase originale a été transformée en nouvelles variantes. La Data Augmentation peut être utile lorsque le dataset est petit, car elle permet d'augmenter le nombre d'exemples disponibles pour l'entraînement d'un modèle.

17 Open Source LLMs

Les *Open Source Large Language Models* sont des modèles de langage accessibles publiquement. Ils peuvent être utilisés, étudiés, adaptés et parfois exécutés localement selon leur licence.

17.1 Objectif

L'objectif est de tester un modèle open source disponible sur Hugging Face afin de générer une réponse en français.

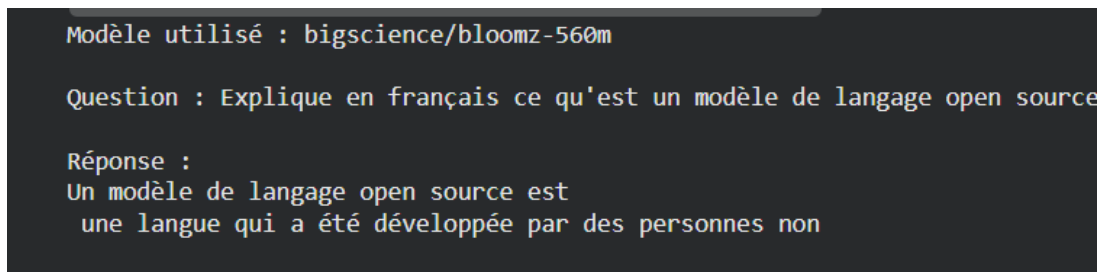
17.2 Code source

```

1 from transformers import AutoTokenizer, AutoModelForCausalLM
2 import torch
3
4 model_name = "bigscience/bloomz-560m"
5
6 tokenizer = AutoTokenizer.from_pretrained(model_name)
7 model = AutoModelForCausalLM.from_pretrained(model_name)
8
9 prompt = ""
10 Question : Explique en français ce qu'est un modèle de langage
    open source.
11
12 Réponse :
13 Un modèle de langage open source est
14 ""
15

```

```
16 inputs = tokenizer(prompt, return_tensors="pt")
17
18 with torch.no_grad():
19     outputs = model.generate(
20         **inputs,
21         max_new_tokens=80,
22         do_sample=False,
23         repetition_penalty=1.3,
24         no_repeat_ngram_size=3,
25         pad_token_id=tokenizer.eos_token_id
26     )
27
28 answer = tokenizer.decode(outputs[0], skip_special_tokens=True)
29
30 print("Modèle utilisé :", model_name)
31 print(answer)
```



```
Modèle utilisé : bigscience/bloomz-560m

Question : Explique en français ce qu'est un modèle de langage open source

Réponse :
Un modèle de langage open source est
une langue qui a été développée par des personnes non
```

FIGURE 24 – Utilisation d'un modèle de langage open source

17.3 Interprétation

La sortie montre l'utilisation du modèle `bigscience/bloomz-560m`. La réponse générée n'est pas parfaite, mais elle montre qu'il est possible d'utiliser un modèle open source pour produire du texte. La qualité de la réponse dépend fortement de la taille du modèle, de son entraînement et du prompt utilisé.

18 Prompt Engineering

Le *Prompt Engineering* consiste à formuler correctement les instructions données à un modèle de langage. Un prompt précis permet généralement d'obtenir une réponse plus claire, plus structurée et plus adaptée au besoin.

18.1 Objectif

L'objectif de cette partie est de comparer deux prompts :

- un prompt simple ;
- un prompt détaillé avec des contraintes.

18.2 Code source

```
1 from transformers import AutoTokenizer, AutoModelForCausalLM
2 import torch
3
4 model_name = "Qwen/Qwen2.5-0.5B-Instruct"
5
6 tokenizer = AutoTokenizer.from_pretrained(model_name)
7 model = AutoModelForCausalLM.from_pretrained(model_name)
8
9 def generate_qwen(prompt, max_new_tokens=180):
10     messages = [
11         {"role": "user", "content": prompt}
12     ]
13
14     text = tokenizer.apply_chat_template(
15         messages,
16         tokenize=False,
17         add_generation_prompt=True
18     )
19
20     inputs = tokenizer(text, return_tensors="pt")
21
22     with torch.no_grad():
23         outputs = model.generate(
24             **inputs,
25             max_new_tokens=max_new_tokens,
26             do_sample=False,
27             temperature=0.1,
28             pad_token_id=tokenizer.eos_token_id
29         )
30
31     generated_ids = outputs[0][inputs["input_ids"].shape[-1]:]
32     answer = tokenizer.decode(generated_ids, skip_special_tokens=
33         True)
34
35     return answer.strip()
36
37 prompt_simple = "Explique le NLP."
38
39 prompt_detailed = """
40 Explique le NLP en français simple.
41
42 Contraintes :
43 1. Donne une définition courte.
44 2. Donne deux exemples d'applications.
45 3. Termine par une conclusion courte.
46 4. Ne répète pas la question.
47 """
48
49 result_simple = generate_qwen(prompt_simple, max_new_tokens=120)
```

```

50 result_detaill   = generate_qwen(prompt_detaill  , max_new_tokens
    =220)
51
52 print("=== Prompt simple ===")
53 print(prompt_simple)
54 print("\nR  ponse obtenue :")
55 print(result_simple)
56
57 print("\n" + "=" * 90 + "\n")
58
59 print("=== Prompt d  taill   ===")
60 print(prompt_detaill  )
61 print("\nR  ponse obtenue :")
62 print(result_detaill  )

```

```

=== Prompt simple ===

Prompt :
Explique le NLP.

R  ponse obtenue :
NLP (Natural Language Processing) est une discipline qui se concentre sur la compr  hension et
l'analyse de la langue humaine. Cela inclut des techniques pour analyser, comprendre et produire la
parole, comme par exemple : 1. Analyse de la parole : Cette partie du langage comprend les mots,
les phrases, les expressions et autres   l  ments linguistiques. 2. Compr  hension de la parole : Elle
consiste    comprendre ce que le linguiste entend par la parole. 3. Produit de la parole : Ceux qui
produ

=====

=== Prompt d  taill   ===

Prompt :
Explique le NLP en fran  ais simple. Contraintes : 1. Donne une d  finition courte. 2. Donne deux
exemples d'applications. 3. Termine par une conclusion courte. 4. Ne r  p  te pas la question.

R  ponse obtenue :
NLP (Natural Language Processing) est l'art de traduire et analyser les langues humaines, en
particulier les langues courantes, pour produire des informations significatives. C'est un domaine
tr  s important dans la technologie et la science, notamment dans la compr  hension du langage humain,
la reconnaissance automatique des signaux, la traduction, etc. Exemple 1 : La traduction
automatique de texte en fran  ais. Exemple 2 : L'intelligence artificielle pour comprendre et
interpr  ter les signes faciaux humains. Conclusion : NLP est essentiel pour am  liorer la
compr  hension et la communication digitale, augmenter la productivit  , et aider    r  soudre des
probl  mes complexes li  s aux langages humains.

=====

```

FIGURE 25 – Comparaison entre un prompt simple et un prompt d  taill  

18.3 Interpr  tation

La comparaison montre que le prompt d  taill   donne une r  ponse plus structur  e. Il impose au mod  le de fournir une d  finition, des exemples et une conclusion. Le prompt simple donne une r  ponse plus libre, mais moins contr  l  e. Cette exp  rience montre que la qualit   de la r  ponse d  pend    la fois du mod  le utilis   et de la pr  cision du prompt.

19 Section Finale

À travers ce TP, nous avons étudié plusieurs notions importantes du NLP. Nous avons commencé par la représentation numérique du texte, puis nous avons abordé les embeddings, les modèles pré-entraînés, les bases vectorielles et le RAG.

Les expériences réalisées montrent que les embeddings permettent de représenter les textes sous forme de vecteurs et de mesurer leur similarité. Les bases vectorielles permettent ensuite de rechercher rapidement les documents les plus proches d'une question.

Nous avons également simulé un système RAG et un chatbot simple. Ces deux exemples montrent comment la recherche documentaire peut améliorer les réponses générées par un système NLP.

Enfin, nous avons testé la Data Augmentation, les modèles open source et le Prompt Engineering. Ces parties montrent que la qualité d'un système NLP dépend non seulement des modèles utilisés, mais aussi des données, du contexte et des instructions fournies au modèle.

20 Projets Étudiants

Plusieurs projets peuvent être réalisés à partir des notions étudiées dans ce TP.

20.1 Projet 1 : Analyse de sentiments

Ce projet consiste à développer un modèle capable de classer automatiquement des textes selon leur sentiment : positif, négatif ou neutre. Les données peuvent provenir de tweets, d'avis clients ou de commentaires.

20.2 Projet 2 : Chatbot intelligent

Ce projet consiste à créer un chatbot capable de répondre aux questions des utilisateurs. Le chatbot peut utiliser une architecture RAG afin de répondre à partir d'une base documentaire.

20.3 Projet 3 : Résumeur automatique

Ce projet consiste à créer une application capable de résumer automatiquement des textes longs. Cette application peut être utilisée pour résumer des articles, des rapports ou des documents pédagogiques.

20.4 Projet 4 : Moteur de recherche sémantique

Ce projet consiste à créer un moteur de recherche basé sur les embeddings. L'utilisateur pose une question et le système retourne les documents les plus proches sémantiquement.

20.5 Projet 5 : Classification automatique de documents

Ce projet consiste à classer automatiquement des documents selon leur catégorie, par exemple : technologie, santé, sport, économie ou éducation.

21 Conclusion

Ce TP nous a permis de comprendre les étapes principales d'un projet NLP moderne. Les méthodes de représentation textuelle, comme les embeddings, jouent un rôle essentiel dans la compréhension automatique du langage.

Les bases vectorielles et le RAG représentent des approches importantes dans les systèmes actuels d'intelligence artificielle. Elles permettent de relier la recherche d'information et la génération de texte.

Enfin, l'utilisation de modèles open source et le Prompt Engineering montrent qu'il est possible d'améliorer les réponses générées en choisissant un bon modèle et en formulant correctement les instructions.